

ПЕРЕГРУЗКА ОПЕРАТОРОВ

Семинар 13

Перегрузка операторов (operator overloading)

- возможность применять встроенные операторы языка к разным типам, в том числе и пользовательским;
- повышает читаемость кода, делает его более лаконичным и выразительным, упрощает написание обобщенного кода;
- в C++ перегрузка операторов имеет серьезную поддержку и активно используется в стандартной библиотеке.

Перегружаемые операторы

Operators that can be overloaded	Examples
Binary Arithmetic	+, -, *, /, %
Unary Arithmetic	+, -, ++, --
Assignment	=, +=, *=, /=, -=, %=
Bitwise	&, , <<, >>, ~, ^
De-referencing	(>)
Dynamic memory allocation, De-allocation	New, delete
Subscript	[]
Function call	()
Logical	&, , !
Relational	>, <, ==, <=, >=

Если не рассматривать операторы управления памятью (new, new[], delete, delete[]), то условно остальные можно разделить на следующие типы:

- унарные (один параметр)
- бинарные (два параметра)
- операторы +, -, *, &, ++, -- (оба варианта)
- оператор () (произвольное число параметров)

Перегружаемые операторы

- Необходимо учитывать приоритет и ассоциативность операторов
- Не рекомендуется перегружать следующие три бинарных оператора: `,` (запятая), `&&`, `||`. Стандарт для них предусматривает порядок вычислений (слева направо), а для последних двух еще и так называемую семантику быстрых вычислений (short-circuit evaluation), но для перегруженных операторов это уже не гарантируется.
- Не рекомендуется перегружать унарный оператор `&` (взятие адреса). Тип с перегруженным оператором `&` опасно использовать с шаблонами, так как они могут использовать стандартную семантику этого оператора.

Реализация перегрузки операторов

Операторы можно перегружать в двух вариантах: как функцию-член и как свободную (не-член) функцию.

Четыре оператора можно перегрузить только как функцию-член — это `=`, `->`, `[]`, `()`.

Для перечислений, операторы можно перегружать только как свободные функции.

Операторы `<<` и `>>` нельзя перегрузить через функцию-член.

Перегрузка оператора как функция-член

- объявить нестатическую функцию-член
 `return_type operator#()`
- в случае перегрузки унарного оператора эта функция не должна иметь параметров,
- в случае бинарного должна иметь ровно один параметр,
- в случае перегрузки оператора `()` эта функция может иметь произвольное число параметров.

```
1  #include <iostream>
2
3  class Complex
4  {
5  public:
6      Complex(int r = 0, int i = 0)
7      {
8          real = r;
9          img = i;
10     }
11
12     // return_type operator#{operands}
13     Complex operator+(const Complex &obj)
14     {
15         Complex res;
16         res.real = real + obj.real;
17         res.img = img + obj.img;
18         return res;
19     }
20
21     void print()
22     {
23         std::cout << real << " " << img << std::endl;
24     }
25
26 private:
27     int real, img;
28 };
```

Перегрузка оператора как функция-член

```
30 ∨ int main(int argc, char **argv)
31 {
32     Complex comp1(4, -2), comp2(3, 5);
33     Complex comp3 = comp1 + comp2;
34     comp3.print();
35     return 0;
36 }
```

Вызов на 33 строке можно представить следующим образом:

`comp1.operator+(comp2)`

```
1  #include <iostream>
2
3  ∨ class Complex
4  {
5  public:
6      Complex(int r = 0, int i = 0)
7      {
8          real = r;
9          img = i;
10     }
11
12     // return_type operator#{operands}
13     ∨ Complex operator+(const Complex &obj)
14     {
15         Complex res;
16         res.real = real + obj.real;
17         res.img = img + obj.img;
18         return res;
19     }
20
21     ∨ void print()
22     {
23         std::cout << real << " " << img << std::endl;
24     }
25
26 private:
27     int real, img;
28 };
```

Перегрузка оператора как функция-член

```

1 #include <iostream>
2
3 class Rational
4 {
5 public:
6     /*конструктор сознательно не explicit, допускает неявное преобразование int в Rational*/
7     Rational(int number = 0, int denominator = 1);
8
9     int numerator() const
10    {
11        return _numerator;
12    }
13
14    int denominator() const
15    {
16        return _denominator;
17    }
18
19    const Rational operator*(const Rational& rhs) const
20    {
21        Rational res;
22        res._numerator = _numerator * rhs._numerator;
23        res._denominator = _denominator * rhs._denominator;
24        return res;
25    }
26
27 private:
28     int _numerator;
29     int _denominator;
30 };

```

shulginaali@O75-Ashulgina:~/013\$ g++ Rational.cpp

Rational.cpp: In function 'int main(int, char**)':

Rational.cpp:46:16: error: no match for 'operator*' (operand types are 'int' and 'Rational')

```

46 |     result = 2 * oneHalf; // ошибка -> 2.operator*(oneHalf)
    |               ^
    |               |
    |             int Rational

```

```

32 int main(int argc, char **argv)
33 {
34     Rational oneEight(1, 8);
35     Rational oneHalf(1, 2);
36
37     Rational result = oneEight * oneHalf;
38     result = result * oneEight;
39
40     result = oneHalf * 2; // -> oneHalf.operator*(2)
41     /*
42     Происходит неявное преобразование типа.
43     Компилятор знает, что вы передали int, а функция требует Rational,
44     но он также знает, что можно получить подходящий объект,
45     если вызвать конструктор Rational с переданным вами аргументом int.
46     Так компилятор и поступает.
47     Конечно, компилятор делает это только потому, что есть конструктор,
48     объявленный без квалификатора explicit.
49     Если бы квалификатор explicit присутствовал,
50     то ни один из вариантов и целым числом 2 не скомпилировался
51     */
52     result = 2 * oneHalf; // ошибка -> 2.operator*(oneHalf)
53     return 0;

```


Перегрузка оператора как функция-член

Если преобразование типов должно быть применимо ко всем параметрам функции (включая и скрытый параметр `this`), то функция не должна быть членом класса.

```
32 int main(int argc, char **argv)
33 {
34     Rational oneEight(1, 8);
35     Rational oneHalf(1, 2);
36
37     Rational result = oneEight * oneHalf;
38     result = result * oneEight;
39
40     result = oneHalf * 2; // -> oneHalf.operator*(2)
41     /*
42     Происходит неявное преобразование типа.
43     Компилятор знает, что вы передали int, а функция требует Rational,
44     но он также знает, что можно получить подходящий объект,
45     если вызвать конструктор Rational с переданным вами аргументом int.
46     Так компилятор и поступает.
47     Конечно, компилятор делает это только потому, что есть конструктор,
48     объявленный без квалификатора explicit.
49     Если бы квалификатор explicit присутствовал,
50     то ни один из вариантов и целым числом 2 не скомпилировался
51     */
52     result = 2 * oneHalf; // ошибка -> 2.operator*(oneHalf)
53     return 0;
54 }
```

Операторы << и >>

```
std::ostream& operator<<(std::ostream &out, const Complex &obj);
```

- При перегрузке через метод класса в качестве левого операнда используется текущий объект. В этом случае левым операндом является объект типа `std::ostream`.
- Мы не можем использовать `std::ostream` в качестве левого неявного параметра, на который бы указывал скрытый указатель `*this`, так как указатель `*this` может указывать только на текущий объект текущего класса, члены которого мы можем изменить, поэтому перегрузка оператора `<<` должна осуществляться через дружественную функцию.

Перегрузка оператора как свободная (не-член) функция

```
1  #include <iostream>
2
3  class Complex
4  {
5  public:
6      Complex(int r = 0, int i = 0)
7      {
8          real = r;
9          img = i;
10     }
11
12     void print()
13     {
14         std::cout << real << " + j" << img << std::endl;
15     }
16
17     friend Complex operator+(const Complex &robj, const Complex &lobj);
18
19 private:
20     int real, img;
21 };
22
23 Complex operator+(const Complex &robj, const Complex &lobj)
24 {
25     return Complex(robj.real + lobj.real, robj.img + lobj.img);
26 }
```

- в случае перегрузки унарного оператора, эта функция должна иметь один параметр,
- в случае бинарного должна иметь два параметра,
- в случае перегрузки бинарного оператора — по крайней мере один из двух параметров, а в случае унарного единственный параметр, должен быть того же типа (или типа ссылки), что и тип, для которого реализуется перегрузка,
- функция должна находиться в том же пространстве имен, что и тип, для которого реализуется перегрузка.

Пример

```
namespace N
{
    class X
    {
        // ...
        X operator+() const;           // унарный плюс
        X operator+(const X& x) const; // бинарный плюс
        void operator()(int x, int y); // вызов функции
        char operator[](int i);       // индексатор
    };
    X operator-(const X& x);           // унарный минус
    X operator-(const X& x, const X& y); // бинарный минус
}
```

Перегрузка операторов

- для унарных операторов используйте перегрузку через методы-класса;
- для бинарных операторов, которые изменяют левый операнд (`operator+=` и т.д.), используйте перегрузку через методы класса, если это возможно;
- для бинарных операторов, которые не изменяют левый операнд (`operator +()` и т.д.), используйте перегрузку через обычные/дружественные функции.

Подробнее

- <https://habr.com/ru/post/489666/>
- <https://en.cppreference.com/w/cpp/language/operators>
- <https://www.geeksforgeeks.org/operator-overloading-cpp/>

Задача

Написать класс `Complex`, который представляет комплексное число в виде действительной и мнимой частей.

Реализовать конструкторы по умолчанию, с параметрами и копирования.

Реализовать перегрузку операторов `+`, `-`, `*` (также для вещественных чисел) и `/` для выполнения арифметических операций с комплексными числами в виде свободных функций.

Реализовать перегрузку операторов `+=`, `-=`, `*=` и `/=` как функций-методов класса.

Реализовать перегрузку оператора `<<` для вывода комплексного числа в поток.